

Student Performance & Attendance Analyzer (SPA)

DBMS Mini Project Implementation & Design Report | Course Code: CS303

GitHub Repository: <https://github.com/smaran-jaianand/MySqlLMS.git>

1. Project Overview & Architecture

The Student Performance & Attendance Analyzer (SPA) is a software tool created to manage academic records and track student performance. The system uses a MySQL database backend and has two interfaces: a web portal and a command-line interface (CLI). The application serves three main types of users: Administrators, Faculty (Professors), and Students. It allows admins to manage accounts, faculty to assign marks and mark attendance, and students to view their reports. The CLI version is packaged as a standalone program that handles database setup automatically on launch.

2. Database Design Choices

I made several important database design choices to ensure data safety and keep the database clean:

- **Database Normalization (3NF):** I split the data into separate tables: *users* for admin logins, *students* for student records, and *faculty* for teacher details. I used junction tables like *student_subjects* (to link students to courses and store marks) and *faculty_subjects* (to link teachers to courses). Attendance records are stored in the *attendance* table, linking back to subjects and students.
- **Cascades (Referential Integrity):** I set up foreign keys with ON DELETE CASCADE and ON UPDATE CASCADE. This means if a student account or subject is deleted, all their corresponding marks and attendance sheets are automatically cleaned up to prevent orphaned records in the database.
- **Checks and Constraints:** I added constraints to prevent incorrect data from being entered. For example, student PRN IDs must match a strict 11-digit pattern, marks must be between 0 and 100, and password hashes must be exactly 64 characters (SHA-256).
- **Rolling Averages:** Instead of recalculating a student's average marks with a slow database query every time, I added an average marks column to the *students* table. The program updates this column automatically whenever a subject mark changes.
- **Database Views:** I created a SQL view called *vw_department_leaderboard* to group student scores by department and calculate averages, highest marks, and lowest marks, making it easy to fetch reports.

3. Challenges I Faced & Solutions

Challenge 1: Handling Database Connection Failures (Authentication Errors)

When I was testing the application, I ran into a major issue where the Python script could not connect to my local MySQL server. MySQL 8 uses a default login security plugin called *caching_sha2_password*. The official *mysql-connector-python* library kept crashing because it lacks built-in support for this plugin on certain Python environments unless extra cryptography packages are installed. I did not want to force users to manually run complicated setups or alter their MySQL root user password settings.

How I solved it: I wrote a smart database connection function. It first tries to connect using the standard *mysql-connector-python*. If it fails with an authentication error, it automatically falls back and connects using the *PyMySQL* library, which supports the login plugin natively in pure Python. This made the application run smoothly on different setups without any manual configuration.

Challenge 2: Keeping Student Average Marks Correct Across Tables

I wanted to save the student's overall average mark in the *students* table so it could be displayed quickly. However, I realized this creates a sync issue: if a faculty member adds or changes a student's mark in a specific subject (which is stored in a separate *student_subjects* junction table), the overall average in the *students* table becomes outdated.

How I solved it: I created a function called *update_overall_student_marks*. Every time a faculty member updates a student's marks in a subject, this function automatically runs. It uses a subquery to calculate the average of all marks for that student in the *student_subjects* table and immediately updates the *students* table with the new result. This keeps all tables in sync.

4. AI Tool Assistance

I used AI tools to help me with a few specific parts of the project:

- **Regex Constraints:** I used AI to write the regular expression pattern `^(25|26|27)0(06|07)(01|02|03|04|05|06)[123][0-9]{3}$` to validate student PRNs during registration.
- **Error Handling:** I used AI to help brainstorm try-except code blocks to catch database connection issues (like wrong passwords or server timeouts) and print clear error messages instead of crashing the program.
- **SQL Views Syntax:** I consulted AI to check the SQL syntax for the complex views, especially when grouping and calculating percentages for attendance logs.